

Enhancing Cloud Network Resilience via a Robust LLM-Empowered Multi-Agent Reinforcement Learning Framework

Yixiao Peng, Hao Hu, Feiyang Li, Xinye Cao, Yingchang Jiang,
Jipeng Tang, Guoshun Nan, *Member, IEEE*, and Yuling Liu, *Member, IEEE*

Abstract—While virtualization and resource pooling empower cloud networks with structural flexibility and elastic scalability, they inevitably expand the attack surface and challenge cyber resilience. Reinforcement Learning (RL)-based defense strategies have been developed to optimize resource deployment and isolation policies under adversarial conditions, aiming to enhance system resilience by maintaining and restoring network availability. However, existing approaches lack robustness as they require retraining to adapt to dynamic changes in network structure, node scale, attack strategies, and attack intensity. Furthermore, the lack of Human-in-the-Loop (HITL) support limits interpretability and flexibility. To address these limitations, we propose CyberOps-Bots, a hierarchical multi-agent reinforcement learning framework empowered by Large Language Models (LLMs). Inspired by MITRE ATT&CK’s “Tactics-Techniques” model, CyberOps-Bots features a two-layer architecture: (1) An upper-level LLM agent with four modules—ReAct planning, IPDRR-based perception, long-short term memory, and action/tool integration—performs global awareness, human intent recognition, and tactical planning; (2) Lower-level RL agents, developed via heterogeneous separated pre-training, execute atomic defense actions within localized network regions. This synergy preserves LLM adaptability and interpretability while ensuring reliable RL execution. Experiments on real cloud datasets show that, compared to state-of-the-art algorithms, CyberOps-Bots maintains network availability 68.5% higher and achieves a 34.7% jumpstart performance gain when shifting the scenarios without retraining. To our knowledge, this is the first study to establish a robust LLM-RL framework with HITL support for cloud defense.

Index Terms—Cloud Network, Cyber Resilience, Adaptive Defense, Multi-agent Reinforcement Learning, Large Language Model, Human-in-the-Loop.

I. INTRODUCTION

With the widespread adoption of cloud computing technologies, cloud networks have become critical infrastructure

This work was supported in part by the National Natural Science Foundation of China under Grants 61902427 and 62471064, and by the National Key Research and Development Program of China under Grants 2023YFC3306305 and 2022YFB2902200. (Corresponding author: Yuling Liu.)

Y. Peng, H. Hu, F. Li, Y. Jiang, and J. Tang are with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, China, and also with the Henan Key Laboratory of Information Security, Zhengzhou, China (e-mail: 13730818683@163.com; wjjhh_908@163.com; lfyxxgcdx@163.com; yingchangjiang@163.com; 19138037519@163.com).

X. Cao and G. Nan are with the National Engineering Research Center for Mobile Network Technologies, Beijing University of Posts and Telecommunications, China (e-mail: caoxinye@bupt.edu.cn; nanguo2021@bupt.edu.cn).

Y. Liu is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China (e-mail: liuyuling@iie.ac.cn).

Yixiao Peng and Hao Hu are equally contributed.

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

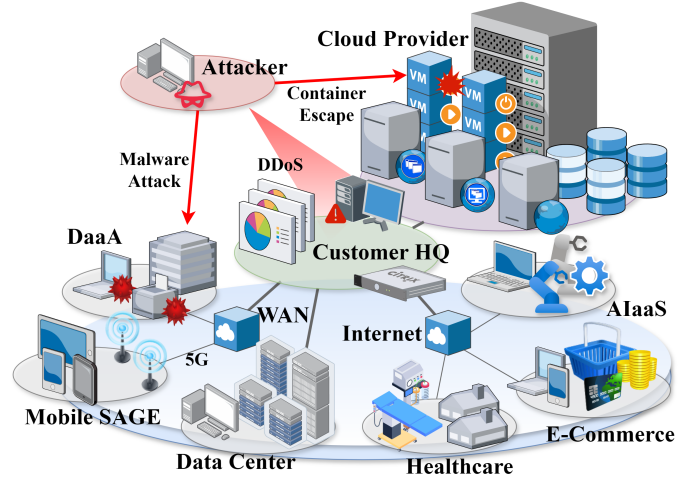


Fig. 1. While technologies like virtualization and elastic scaling provide dynamic flexibility, they inevitably expand the attack surface. This increased exposure facilitates specific cloud attacks, such as container escape and malware in cloud storage, thereby challenging system resilience.

supporting various information services [1], [2]. Their applications have penetrated numerous sectors including government, finance, industry, and healthcare [3], [4]. Cloud networks exhibit highly dynamic characteristics: virtualization and elastic scaling mechanisms lead to frequent creation, destruction, and reconfiguration of network components like virtual machines and container instances, resulting in continuous changes in network structure and scale [3]. While enhancing resource utilization efficiency and system flexibility, cloud networks also present a large-scale, distributed attack surface, making them ideal targets for coordinated attacks and posing severe challenges to network resilience [5]. Consequently, Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) have been widely applied in cyber security decision-making due to their ability to autonomously learn optimal defense strategies through environmental interaction [6], [7]. The core principle involves modeling the attack-defense process as a Markov Decision Process and solving for the optimal policy by maximizing cumulative rewards, thereby balancing defense costs with network availability to enhance cyber resilience. However, the dynamic nature of cloud networks and the threats they face impose adaptability challenges on existing DRL-based network defense decision-making methods.

Specifically, as illustrated in Fig.2, the cloud-native architecture of a large e-commerce platform serves as a typical example of a cloud network. It consists of virtual machine

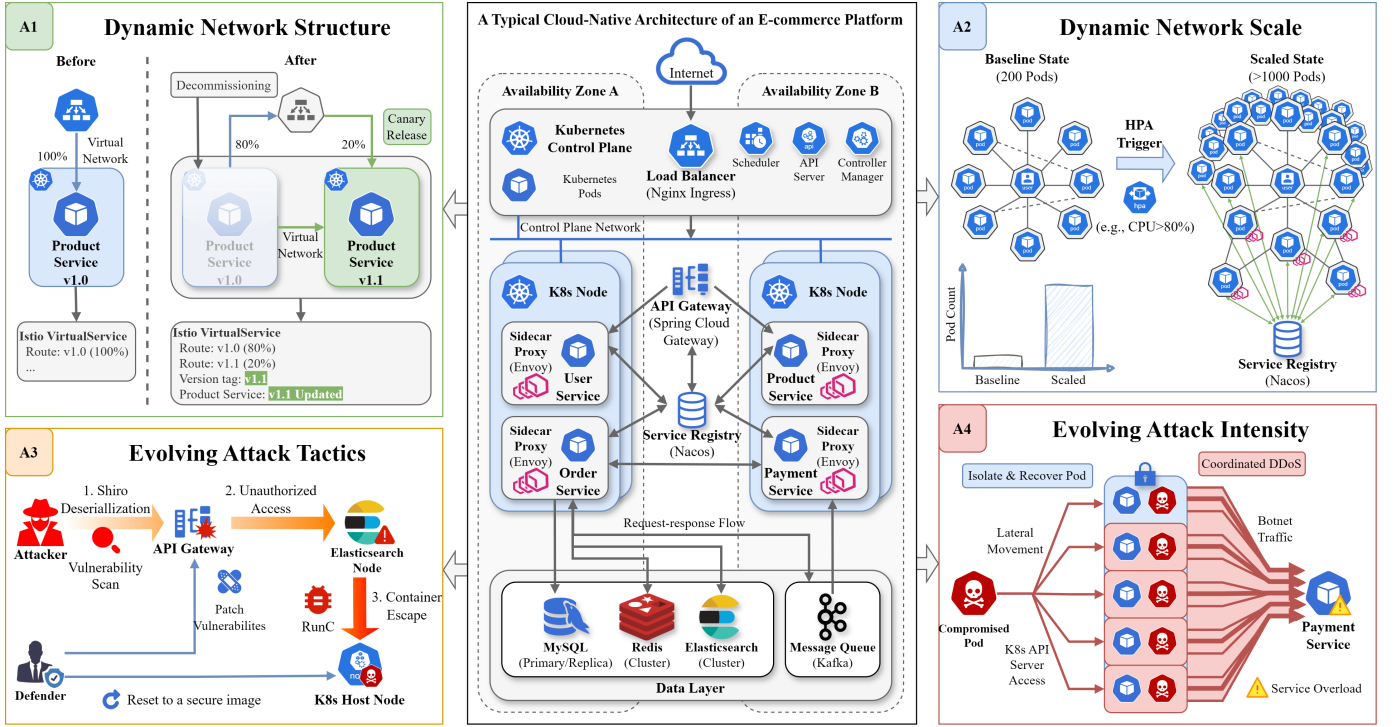


Fig. 2. A typical cloud-native e-commerce architecture, exemplifying the four dynamic aspects (A1-A4). i) The network frequently performs elastic scaling and instance migration in response to workload fluctuations. ii) Business instances dynamically expand as new features are deployed or partners integrated. iii) Meanwhile, attack tactics and scale are constantly evolving, from initial port scanning of public entry points, to compromising databases via vulnerable middleware, and ultimately escalating into coordinated DDoS attacks that cripple core services.

clusters across multiple Availability Zones, containerized microservices, load balancers, and distributed database instances. Due to fluctuating business loads, this network frequently undergoes elastic scaling and instance migration, leading to continuous changes in the virtual network topology and east-west traffic policies (A1). Concurrently, business modules dynamically scale the number of instances as new features are deployed or partners are integrated, resulting in a highly dynamic number of network nodes (A2). Meanwhile, attack tactics and scales are constantly evolving: an attack may start with port scanning and service probing on public-facing entry points, then compromise database instances through vulnerable middleware (A3) [8], [9], and finally escalate into coordinated attacks aiming to paralyze core services via encryption ransomware or DDoS flooding (A4) [10]. To counter such threats, defenders can take various actions, such as resetting compromised instances to clean images, isolating compromised container nodes to contain spread, patching microservice vulnerabilities to eliminate attack surfaces, or restoring isolated nodes to maintain business continuity.

Therefore, we define a **robust cloud network defense decision-making framework** as one that can seamlessly adapt to four key dynamic aspects (A1-A4) while maintaining defense effectiveness without the need for retraining:

- **A1: Adaptability to Network Structure.** Alibaba Cloud cluster tracking data shows that 11,089 container rescheduling and co-locating events can occur within 12 hours [11], indicating extremely high network structure dynamics. Therefore, when network configurations and

node connections change, the defense decision-making framework should seamlessly adapt without retraining.

- **A2: Adaptability to Network Scale.** According to Google Cloud cluster statistics, the node scale can expand to over ten thousand [12]. Therefore, the defense mechanism should effectively scale to networks containing a large number of diverse node types, avoiding state space explosion.
- **A3: Adaptability to Attack Policy.** The Capital One data breach incident [13] combined multiple techniques such as Server-Side Request Forgery (SSRF) vulnerabilities, AWS instance metadata service abuse, IAM credential theft, and S3 bucket enumeration, reflecting the multi-stage nature of cloud attacks. Therefore, the defense system should dynamically respond to diverse, multi-stage attack strategies.
- **A4: Adaptability to Attack Scale.** Google Cloud Armor statistics indicate that attack traffic targeting a single user can originate from 5,256 source IPs across 132 countries, demonstrating extremely high concurrency [14]. Therefore, when the number of concurrent attackers increases, the defense strategy should maintain robustness and simultaneously handle multiple attack paths.

Here, we clarify the specific meaning of “no retraining” in the context of this work. Specifically, this term indicates that the framework does not require online gradient updates or parameter fine-tuning during inference to maintain defensive performance when encountering the aforementioned dynamics.

Existing research struggles to simultaneously meet the

forementioned adaptability requirements due to two root causes. First, current defense models typically encode network states into fixed-dimensional vectors. For instance, Purves et al. [15] integrate network attributes such as node connectivity, vulnerability CVSS scores, and critical assets into a fixed-length vector, causing the internal parameters (e.g., neural network input tensor shape, weights) to be deeply coupled with the specific network scale (A2) and structure (A1) during training. Once the network environment changes, the state vector dimensionality alters, often necessitating resource-intensive retraining. Second, existing approaches usually employ similar or identical attack patterns and scales during both training and testing phases. For example, [15]–[17] utilize exactly the same attack policies in both phases. Consequently, when confronted with unseen, phase-evolving attack policies (A3) or concurrent attacks of significantly larger scale (A4), the models fail to generalize effectively.

Given this, we propose CyberOps-Bots (Cyber Operation Bots), a robust hierarchical multi-agent framework based on Large Language Models (LLMs) and RL, designed to build an agent cluster for automated collaborative cyber defense operations in cloud networks. Inspired by the MITRE ATT&CK framework [18], the framework adopts a “Tactics-Techniques” co-governance design: the upper-layer LLM agent is responsible for global situation awareness, human intent recognition, defensive tactical planning, and resource scheduling, leveraging its semantic understanding and multi-step reasoning capabilities; the lower-layer RL agents execute specific defensive actions within localized network regions, relying on their RL mechanisms for efficient and precise control. Through this hierarchical design, the framework preserves the adaptability and interpretability of LLMs in high-level planning while ensuring the reliability and executability of low-level actions via RL. Furthermore, the framework inherently supports human-in-the-loop (HITL), allowing security experts to inject prior knowledge or intervene in real-time through natural language at the LLM tactical layer, enabling human-machine collaborative defense. The decision-making process is recorded as an auditable, readable tactical planning reasoning chain. By seamlessly integrating human expert domain knowledge and judgment into the automated decision cycle, the system can flexibly adjust strategies in complex, dynamic adversarial environments, compensating for the limitations of purely algorithmic models in semantic understanding, ethical trade-offs, and response to unforeseen events.

It is acknowledged that a centralized LLM planner, while offering interpretability and tactic planning, may introduce new security considerations in deployment, such as becoming a single point of failure or a target for adversarial prompt manipulation and observation poisoning. Addressing these model-level security threats, however, extends beyond the scope of this paper, which focuses on establishing the architectural paradigm and validating its core adaptive capabilities.

The main contributions of this paper are as follows:

- We propose a natural language-based method for representing cyber adversarial scenarios. By converting high-dimensional, structured network state information and network context into natural language descriptions as

input for LLM-based global situational awareness, the defense framework becomes independent of specific network structures. This enables seamless adaptation to changes in network topology (A1) and scale (A2).

- We design a hierarchical decision-making framework combining LLM and RL suitable for large-scale network environments. The upper-layer LLM agent performs global tactical planning using the ReAct paradigm, while lower-layer RL agents execute atomic defense actions within localized network regions. This effectively mitigates the state space explosion problem caused by network scaling (A2).
- We introduce a heterogeneous separated pre-training architecture. By designing specialized reward functions and training scenarios, a set of functionally heterogeneous RL expert agents are trained offline. The LLM then performs semantic-level tactical parsing and planning based on real-time attack-defense situations, scheduling different RL agents via tool invocation to form adaptive defense strategies tailored to varying attack policies (A3).
- We develop a long-short term memory mechanism for multi-stage, multi-path attacks. This mechanism stores multiple attack chains and retrieves them based on similarity matching, allowing the LLM to track the evolution of multiple attack chains across time steps and conduct defense using “reactive” memory and tool calls. This enhances the framework’s capability to handle increased concurrent attack scales (A4).

II. RELATED WORKS

Cloud networks, characterized by dynamic scheduling and scalability, face significant threats from multi-staged attacks. While technical approaches exist, they struggle to simultaneously adapt to changes in network structure (A1), scale (A2), attack policy (A3), and intensity (A4).

RL-based methods model cyber-defense as MDPs or game-theoretic processes [6], [7], [26]–[28] and learn strategies within the models through RL algorithms. These have been applied to CPS [29], [30], edge computing [31], and IoT [32], [33]. To enhance decision-making, researchers have employed MARL [34], [35] and MADRL [16], [36] against APTs. However, these methods often require retraining or suffer from policy failure when network scales expand or attack phases shift.

To address the adaptability limitations, numerous works have proposed adaptive defense frameworks.

In terms of enhancing adaptability to attack strategies, Ren et al. [25] employed adversarial training and synchronous interaction mechanisms. While promising, their experimental validation focused on fixed network and attacker scales. Cao et al. [24] introduced a self-evolving Moving Target Defense (MTD) approach aimed at overcoming the generalization challenges arising from discrepancies between training environments and attack-defense scenarios. However, evaluations of this method were mainly conducted on a fixed Software-Defined Networking (SDN) topology. Purves et al. [15] incorporated a causal inference model to replace traditional reward

TABLE I
COMPARISON OF EXISTING DEFENSE METHODS AGAINST ROBUSTNESS REQUIREMENTS.

Ref.	Robust Adaptability				Framework	Scenario	Resilience Aware	Attack Policy	Interpretability	HITL Support
	A1	A2	A3	A4						
[19]	•	◦	•	•	LLM-RL-KI	Cloud storage	Yes	Multi-stage Rule-Based Policies	Based on knowledge vector (Explicit)	Auditable Knowledge Quadruple
[20]	•	•	•	◦	LLM Agent OAPP	IoT	No	-	Based on LLM ReAct reasoning (Explicit)	Customizable Workflow
[21]	•	◦	•	◦	OAPP	5G-ICPS	No	Dual RL Agents	Based on strategy output (Implicit)	No
[17]	•	•	◦	◦	Actor-Critic RL	Smart grids	Yes	Trained Q-Learning	Based on strategy output (Implicit)	No
[22]	◦	•	•	◦	Hierarchical PPO	Cloud computing	Yes	Multiple Static Rule-Based Policies	Interpretable metrics (Implicit)	No
[23]	◦	◦	•	•	GMADRLD	Cloud computing	No	Multiple Policies with Variants	Based on strategy output (Implicit)	No
[15]	•	◦	•	◦	PPO+SCM	Autonomous cyber operation	Yes	Multiple Static Rule-Based Policies	Based on SCM rewards (Implicit)	Limited: SCM Human Knowledge Integration
[24]	◦	•	•	◦	PPO	SDN	Yes	Multiple Static Rule-Based Policies	Based on strategy output (Implicit)	No
[25]	◦	◦	•	◦	DDQN	IoT	No	DQN Agent	Based on strategy output (Implicit)	No
Ours	•	•	•	•	LLM+RL Agents	Cloud network	Yes	Multi-stage Intensity-Escalation Attack	Based on LLM ReAct reasoning (Explicit)	Expert Instructions Integration

models. Nevertheless, the computational complexity of reward based on causal graphs may pose scalability challenges in large-scale networks. Targeting multi-stage APT attacks in cloud computing networks, Chen et al. [23] proposed the GMADRLD algorithm, which enhances resource allocation efficiency by grouping agents. Experiments demonstrated its adaptability to varying attack strategies and scales.

In terms of enhancing adaptability to network scale and structure, Singh et al. [22] employed a hierarchical PPO with a two-stage training method to address the state space explosion problem in decentralized networks. It is worth noting that the training of upper-layer agents in this framework incorporates expert rules, which may involve specific manual adjustments when adapting to different cyber-defense scenarios. While traditional hierarchical RL methods [37] decompose complex tasks into sub-goals, their high-level policies are often constrained by fixed numerical state-action mappings and struggle with cross-topology generalization. In contrast, our LLM-based planner shifts the hierarchical coordination from the numerical level to the semantic level. Xiao et al. [17] focused on APT defense in large-scale smart grids. While their approach demonstrates capability in large-scale node management (each MDMS node connects 50–150 smart meters), the adaptation to diverse attack policies is not discussed. In 5G Industrial Cyber-Physical Systems (ICPS), Li et al. [21] constructed a dual-network model to predict attack paths. However, their model depends on static topology and vulnerability information; when network configurations change, the dual-network model must be reconstructed and retrained.

In recent years, with the advancement of LLMs and LLM Agents [38]–[40], agent-based defense methods have begun to emerge. For example, IDS-Agent [20] constructs an LLM-based agent that integrates multiple model tools and external memory retrieval to achieve explainable intrusion detection in

IoT. LLM4Game [19] is a MARL defense method based on knowledge injection, which uses strategy vectors generated by LLMs to dynamically adjust reward functions. However, due to inherent limitations of LLMs, their in-depth application faces challenges. First, LLMs have limitations in numerical computation [41], as their mathematical reasoning is not based on formal logic but on abstract probabilistic matching, making it difficult to reliably handle high-dimensional, dynamic network state spaces. For instance, LLMs may struggle to accurately interpret node connectivity represented by network adjacency matrices or calculate the shortest distance from attackers to critical assets. Second, LLMs are better at macro-level planning than constrained decision-making [42], making it difficult to directly generate executable defense commands. For example, even if an LLM can generate a tactical intent such as isolating the infected network segment, it may struggle to translate this into valid flow table modification.

Currently, no framework systematically addresses these challenges. To fill this research gap, the proposed CyberOps-Bots innovatively adopts a hierarchical LLM+RL architecture, combining the semantic understanding and planning capabilities of LLMs with the atomic defense action orchestration capabilities of RL in dynamic environments. The upper-layer LLM agent is responsible for understanding the attack-defense situation, tactical planning, and resource scheduling, while the lower-layer RL agents execute specific atomic defense actions. This approach leverages the strengths of LLMs while effectively circumventing their limitations, offering a new paradigm for building adaptive cloud network defense systems.

To clearly demonstrate the coverage of adaptive capabilities in existing research, Table I summarizes the abilities of related works in addressing adaptability challenges A1–A4. As can be seen from the table, there is currently a lack of a complete solution capable of simultaneously meeting all four adaptability

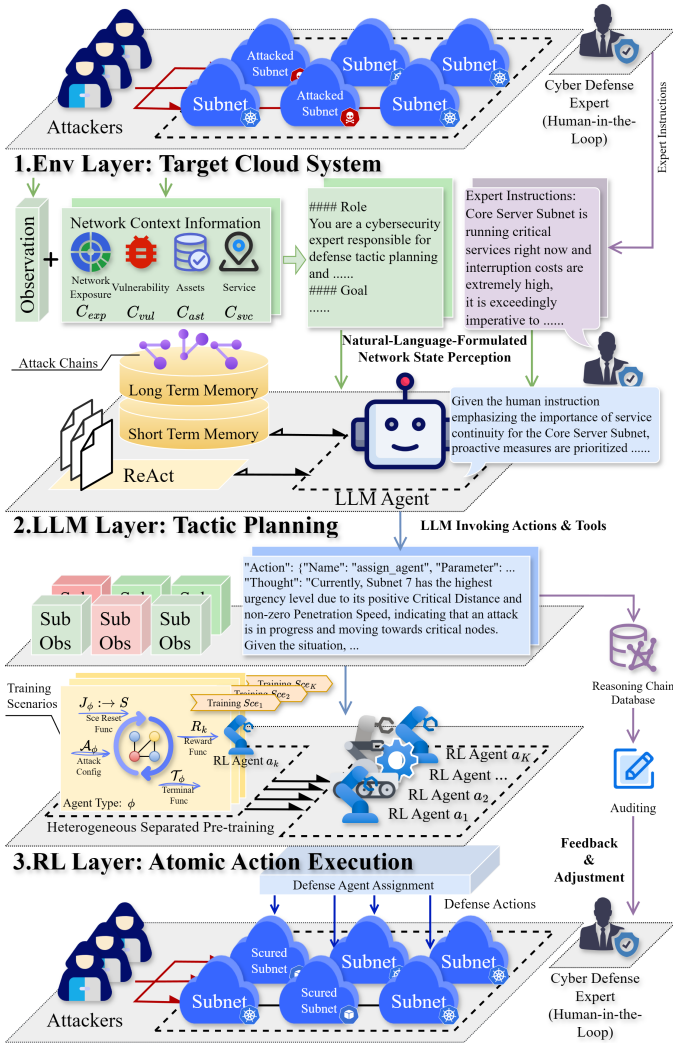


Fig. 3. The CyberOps-Bots framework architecture, comprising three coordinated layers: (i) the Env Layer simulating a dynamic adversarial cloud network; (ii) the LLM Layer with Perception, Planning, Memory, and Action modules for semantic reasoning and HITL support; and (iii) the RL Layer of pre-trained heterogeneous agents executing localized defense actions, enabling adaptive response without retraining.

requirements mentioned above.

III. METHODOLOGY

A. CyberOps-Bots Framework

The overall architecture of the CyberOps-Bots framework is illustrated in Fig.3, which consists of three core layers: the Env Layer, the LLM Layer, and the RL Layer. These correspond respectively to three stages: environmental interaction, tactical planning, and action execution.

The Env Layer simulates a dynamic adversarial scenario in a cloud environment, where attackers perform penetration and lateral movement across multiple virtual subnets, providing a real-time confrontation environment for the agent framework.

The upper-layer LLM agent plays four roles in our framework. First, it drives the perception module, converting high-dimensional structured network states and unstructured contextual information into unified semantic descriptions based

on the NIST IPDRR framework [43]. This perception mode ensures defense decisions do not rely on specific network topologies or scales, enabling inherent adaptation to dynamic changes in network structure (A1) and scale (A2) without retraining. Second, the LLM executes high-level tactical planning with strong generalization. Through the ReAct paradigm, it conducts multi-step reasoning to dynamically orchestrate and schedule heterogeneous lower-layer RL agents via tool calls. This semantic-based planning capability enables flexible handling of unseen multi-stage attacks (A3), rather than merely relying on predefined strategy combinations. Concurrently, through integration with long- and short-term memory mechanisms, the LLM maintains and analyzes attack chains across multiple time steps. It retrieves historical attack patterns via similarity-based matching to infer attack intentions and propagation trends, enabling effective response to changes in attack scales (A4) and “reactive” memory-based tactical planning. Traditional planners lacking semantic association and context preservation do not possess this capability. Finally, as an HITL interface, the LLM integrates expert knowledge and intervention via the perception module. Its auditable reasoning chains incorporate human judgment into the decision cycle, enhancing system trustworthiness in adversarial environments.

The lower-layer RL agents comprise a set of functionally heterogeneous agents. These agents are pre-trained offline in specially constructed scenarios, with each agent responsible only for its assigned segment of the network. This design avoids state space explosion due to dynamic network scale (A2).

B. Framework Formulation

This section models the dynamics of cloud network attack and defense using a MDP and formally defines the proposed hierarchical multi-agent framework, CyberOps-Bots. The MDP process is designed in conjunction with the Yawning Titan [44] cyber attack-defense simulation platform, developed by the UK Defence Science and Technology Laboratory (Dstl). This platform has been widely adopted in academic research and practical cyber exercises, with its simulation environment validated for adversarial complexity and strategy evaluation effectiveness [15], [45].

Definition 1: The CyberOps-Bots framework is formally represented as a quadruple $(LLM, \mathcal{A}_{lower}, M, T)$, where:

- LLM denotes the upper-layer LLM agent, responsible for global tactical planning.
- $\mathcal{A}_{lower} = \{a_1, a_2, \dots, a_K\}$ represents the set of lower-layer RL agents, each executing atomic defense actions within localized network regions.
- M denotes the memory module, comprising short-term and long-term memory.
- T denotes the toolset, which the LLM can call upon to implement tactical plans.

Definition 2: The MDP is defined as a (S, A, P, R, γ) , where:

- S is the state space.
- A is the action space.
- $P : S \times A \times S \rightarrow [0, 1]$ is the state transition probability function.

- $R : S \times A \times S \rightarrow \mathbb{R}$ is the reward function.
- $\gamma \in (0, 1)$ is the discount factor.

1) *State*: This section defines the global state space, the upper-layer observation space, and the lower-layer observation space.

Definition 3: The global state $s_t \in S$ at time t captures the complete security posture of the cloud network and is defined as:

$$s_t = \{C, G_t, h_t(n), q_t(n), \text{vuln}_t, \text{HVN}, \text{Entry}, I_t^{\text{Human}}\}, \quad (1)$$

where:

- C is the network context, which consists of semantic information about network assets, vulnerabilities, and service continuity, organized based on the NIST IPDRR framework [43]. It provides standardized natural language input for the upper-layer LLM agent's situational awareness (see Section III-D2).
- The cloud network is represented by an undirected graph $G_t = (V, E_t)$, whose adjacency matrix is denoted as Adj_t .
- $h_t(n)$ and $q_t(n)$ represent the health status and isolation status of node n , respectively.
- $\text{vuln}_t(n)$ reflects the vulnerability level of node n .
- HVN is a binary vector identifying which nodes are high-value nodes.
- Entry is a binary vector identifying entry nodes accessible from external attacks.
- I_t^{Human} represents prior knowledge or real-time intervention instructions injected by security experts via natural language at time t , which is the core of the HITL mechanism. When there is no human input, $I_t^{\text{Human}} = \emptyset$.

Definition 4: The observation $o_t^{\text{LLM}} \in O^{\text{LLM}}$ of the upper-layer LLM agent is a semantic perception of the global state s_t , generated by the Perception module (detailed in Section III-D2) combining the global state and human instructions.

Definition 5: The local observation $o_t^k \in O_k$ of a lower-layer RL agent a_k at time t is the projection of the global state s_t onto the local network region $G_t' \subseteq G_t$ assigned to that agent (detailed in Section III-C1).

Since the LLM agent perceives the network state in natural language, and each RL agent only perceives local observations, their observation spaces maintain structural and semantic consistency even when the network structure and scale change, thus requiring no retraining for adaptation.

2) *Action*: For the defender, the proposed framework defines two distinct action spaces: a high-level tactical action space A^{LLM} for the upper-layer LLM agent and a low-level atomic action space A^{lower} for the lower-layer RL agents.

Definition 6: The atomic action space A^{lower} consists of fundamental defensive operations applicable to cloud network nodes. An atomic action $act_k \in A^{\text{lower}}$ is defined as a tuple $(operation, target)$, where $operation$ denotes the type of defensive action (e.g., resetting a container image, patching a node vulnerability, isolating a compromised container, or restoring a node's service), and $target$ specifies the node $n \in V$ to which the action is applied.

Definition 7: The tactical action space A^{LLM} for the upper-layer LLM agent comprises the set of tactical instructions

it can execute. This includes both directly invoking atomic actions and calling tools, formally represented as $A^{\text{LLM}} = \{A^{\text{lower}}, \mathcal{T}\}$.

For the attacker, actions are modeled based on penetration capabilities. Attackers can initiate attacks from any compromised node or entry node towards adjacent nodes. The success probability $P_{\text{attack}}(s_t, RS)$ of an attack on a visible target node n_{target} depends on the target node's vulnerability and the attacker's skill level $RS \in [0, 1]$ [15], calculated as:

$$P_{\text{attack}}(s_t, RS) = \min \left(\frac{RS^2}{RS + (1 - \text{vuln}_t(n_{\text{target}}))}, 1 \right). \quad (2)$$

If the attack is successful, the health state of the target node is updated to Compromised: $h_{t+1}(n_{\text{target}}) = \text{Compromised}$.

3) *Global Reward Function*: This section designs the global reward function.

Definition 8: To ensure the reward design aligns with the practical requirements of cloud network defense, the reward objectives are established with reference to the three aspects defined in the ISO/IEC 27001:2022 standard [46]: asset management, network security, and defense continuity. The global reward function R is formulated as:

$$R(s_t, a_t, s_{t+1}) = R_{\text{asset}}(s_{t+1}) + R_{\text{security}}(s_t, s_{t+1}) + R_{\text{cost}}(a_t), \quad (3)$$

where:

a) **Asset Protection Reward (R_{asset})**:

$$R_{\text{asset}}(s_{t+1}) = -\lambda_{hva} \cdot \sum_{n \in V} \text{HVN}_t(n) \cdot h_{t+1}(n). \quad (4)$$

This component penalizes the compromise of high-value nodes (HVNs), aiming to prevent critical data leakage or tamper. A significant penalty is incurred when an HVN is compromised.

b) **Network Security Reward (R_{security})**: This term penalizes all nodes in an abnormal state (e.g., compromised or isolated) within the network. It comprehensively considers both the quantity and the change in abnormal nodes.

$$R_{\text{security}}(s_t, s_{t+1}) = -(\alpha \cdot N_{t+1}^{\text{impact}} + \beta \cdot \Delta N_t^{\text{impact}}). \quad (5)$$

Here, N_{t+1}^{impact} denotes the number of abnormal nodes at time $t+1$, and $\Delta N_t^{\text{impact}}$ represents the number of newly added abnormal nodes from time t to $t+1$. The coefficients α and β are weighting factors.

c) **Action Cost Reward (R_{cost})**:

$$R_{\text{cost}}(a_t) = -\text{Cost}(\text{operation}). \quad (6)$$

C. Lower-Layer RL Agents

The lower layer of the framework comprises multiple types of specially trained RL agents, denoted as the set $\mathcal{A}_{\text{lower}} = \{a_1, a_2, \dots, a_K\}$ in Definition 1. These agents possess diverse defense objectives and specialize in different action types. Through the tactical planning and orchestration by the upper-layer LLM, they are combined to form varied defense strategies, thereby enabling the framework to adapt to different attack policies (A3).

1) *Agent Setting*: For a lower-layer RL agent a_k assigned to a subnet $G'_t \subseteq G_t$ at time step t , its decision-making is based on local observation information within that network segment. This design prevents the state space from exploding as the network scale increases.

Definition 9: The sub-observation space and sub-action space for an agent are subsets of their global counterparts, formally defined as follows:

- a) **Sub-observation space** O_k : Consists of the local network state variables that RL agent a_k focuses on, such as node health status, vulnerability level, isolation status, network topology connections, and high-value node distribution. Its specific composition depends on the agent's defensive expertise and task objectives.
- b) **Sub-action space** A_k : Contains the types of defensive actions executable by agent a_k and their scope of application, typically represented as $A_k = \{(action, n) \mid action \in O_k, n \in G'_t\}$, where $O_k \subseteq operation$ is the set of defensive actions that the agent can perform.

The determination of the above observation and action spaces depends on the function of the corresponding agent. For example, consider a *Block Agent*. This agent is responsible for precisely blocking attack paths by executing the *Isolate* action when attackers approach high-value nodes. Therefore, this agent needs to comprehensively analyze the network topology, node health status, and high-value node distribution to accurately determine isolation targets. Its sub-observation space is defined as $O_{iso} = \{Adj_t, h_t(n), HVN \mid \forall n \in G'_t\}$, and its sub-action space is defined as $A_{iso} = \{(Isolate, n) \mid \forall n \in G'_t\}$.

2) *Heterogeneous Separated Pre-training*: We introduce a heterogeneous separated pre-training framework for lower-layer RL agents, integrating reward function design and scenario construction. This guarantees stable training of specialized agents with differentiated defensive skills, providing the upper-layer tactical planner with diverse defenses to address dynamic attack policies (A3).

Definition 10: To guide the lower-layer RL agents in learning and reinforcing their specific decision-making styles, we design specialized reward functions for each type of agent, forming a reward function set $\mathcal{R}_{lower} = \{R_1, R_2, \dots, R_K\}$. These functions are built upon the global reward function $R(s_t, a_t, s_{t+1})$, but place greater emphasis on the objectives most relevant to their respective subtasks, thereby training functionally heterogeneous expert agents.

This independent training design addresses the issue of training instability in Hierarchical Multi-Agent Reinforcement Learning (HMARL) [22]. It specifically tackles the instability caused by the interdependencies that exist between policy hierarchies, among agents, and across different subtasks during the HMARL training process.

Definition 11: A training scenario for an agent of type ϕ is formally defined as a triple $(\mathcal{I}_\phi, \mathcal{A}_\phi, \mathcal{T}_\phi)$, where:

- $\mathcal{I}_\phi : \rightarrow S$ is the scenario reset function, which generates an initial state s_0 aligned with the learning objectives of agent type ϕ ;
- $\mathcal{A}_\phi$ is the attack configuration, defining the number of attackers and their behavior patterns within the scenario;

- $\mathcal{T}_\phi : S \rightarrow \{0, 1\}$ is the termination condition function.

Through the construction of these training scenarios and the independent training of different RL agents, the method resolves the agent training dependency problem inherent in HMARL [22]. It enables each agent type to learn its policy in an environment highly tailored to its responsibilities, ensuring both training stability and efficiency.

Algorithm 1 Heterogeneous Separated Pre-training.

```

1: Input: Agent type  $i$ , training episode length  $T$ ,
2: Output: Trained policy network parameter  $\theta_i$ 
3: for  $e$  in  $E$  do
4:   Initialize scenario  $S_i^e$ 
5:   Reset environment with  $S_i^e: s_0 \sim \mathcal{R}(S_i^e)$ 
6:   for  $t$  in  $T$  do
7:     Observe:  $o_t^i$ 
8:     Select action with policy  $\pi_{\theta_i}: a_t^i \sim \pi_{\theta_i}(o_t^i)$ 
9:     Execute  $a_t^i$  and attack according to  $A(S_i^e): s_{t+1}, r_t^i \sim Env(s_t, a_t^i)$ 
10:    Calculate reward  $R_i(s_t, a_t^i, s_{t+1})$ 
11:    Append  $(s_t, a_t^i, r_t^i, s_{t+1})$  to buffer  $D$ 
12:    if learn then
13:      Sample a training batch from  $D$ 
14:      Update  $\theta_i$  by minimizing loss
15:    end if
16:  end for
17: end for
18: Return  $\theta_i$ 

```

The heterogeneous separated pre-training method is shown in Algorithm 1, which can be adapted to any single-agent RL algorithm for agent parameter updates.

D. Upper-Layer LLM Agent

The upper layer of the proposed hierarchical framework is an LLM agent. Serving as the global decision-maker, its core function is to integrate the execution capabilities of RL agents with high-level semantic planning to address adaptability challenges in cloud networks.

The LLM agent architecture comprises four core components [40]: Planning, Perception, Memory, and Action and Tools. Specifically, the agent transforms state information into natural language via the Perception module, utilizes the Planning module for multi-step reasoning and tactical formulation, maintains attack chain context through the Memory module, and translates decisions into concrete defensive actions or scheduling instructions for lower-layer RL agents via the Action and Tools module.

The following subsections detail each module.

1) *Planning*: To achieve adaptive planning, the upper-layer LLM agent employs the ReAct [47], [48]. ReAct is a prompting technique that synergizes reasoning and acting. By guiding the LLM to explicitly generate reasoning traces and subsequent actions, it significantly enhances the reliability of decision-making in complex tasks.

At each time step t , the planning process of the LLM agent is as follows:

- a) **Observation:** The LLM receives network situational information from the Perception module, the Long-Term Memory LTM_t and Short-Term Memory STM_t stored in the Memory module, as well as a list of available tools and actions (including their functions, parameters, etc.).
- b) **Reasoning:** Based on the input observation information O_t^{LLM} , the LLM generates a reasoning text. By analyzing the situation and objectives, it formulates a strategy and concludes the reasoning process.
- c) **Acting:** Following the reasoning, the LLM outputs the corresponding action and its parameters, denoted as $(a_t^{LLM}, para_t)$. After the environment executes the action, the state information is updated, new memories are generated, and the cycle repeats.

The reasoning chain generated by the LLM within the ReAct loop is saved as logs, providing an interpretable audit trail for expert review.

2) *Perception:* The perception module acts as the information gateway for the upper-layer LLM agent, converting high-dimensional, structured network state data into natural language. This abstraction decouples defense decisions from specific topologies and scales, ensuring adaptability to structural (A1) and scalar (A2) changes. Crucially, this semantic unification serves as the cornerstone of the framework's robustness, empowering the agent to generalize to unseen network configurations seamlessly without retraining. Additionally, by integrating expert knowledge and human instructions, the module enables a HITL interaction mechanism.

Inputs to this module include the network context C , expert instructions I_t^{Human} , and calculated state metrics. These are aggregated via a prompt template to generate the LLM's observation O_t^{LLM} .

Organized by the NIST IPDRR cybersecurity framework [43], the module processes the following context and metrics:

- a) **Identify:** Identifies potential threat surfaces and propagation risks.
 - Subnet Exposure Surface C_{exp} : Defines connection policies and boundaries between subnets and untrusted networks, identifying the attack surface.
 - Entry Node Count: Quantifies the initial size of the subnet's attack surface.
- b) **Protect:** Evaluates asset vulnerabilities and the effectiveness of existing defenses.
 - Subnet Vulnerability Profile C_{vul} : details device types, firmware, CVEs, and patch status, outlining inherent asset vulnerabilities.
 - Average Vulnerability: Quantifies the subnet's overall vulnerability level.
 - Compromised Node Count: Measures the extent of damage and current defense failure.
- c) **Detect:** Analyzes attack trends, strategies, and patterns.
 - Attack Chain Concentration: Measures strategy concentration via information entropy to identify attack patterns:

$$H_{\text{attack}}(t) = - \sum_{i=1}^k p_i \log(p_i), \quad (7)$$

where p_i is the proportion of attacks on the i -th subnet over time ΔT . Lower entropy indicates a more deterministic strategy.

- Attack Frequency: The average attacks per subnet G' over ΔT , reflecting attacker activity and focus.
- d) **Respond:** Details asset urgency to guide response prioritization.

- HVN Count: The number of HVNs, indicating subnet criticality.
- Critical Distance: The shortest path from compromised nodes to HVNs, reflecting immediate threat urgency. It is defined as:

$$D_t^{\text{critical}}(G') = \min\{\text{DisToHVN}(n_c) \mid n_c \in G', h_t(n_c) = \text{Compromised}\}. \quad (8)$$

- Penetration Speed: The rate of change in Critical Distance, $\Delta D_t^{\text{critical}}(G') = D_{t-1}^{\text{critical}}(G') - D_t^{\text{critical}}(G')$. Positive values indicate attack progression; negative values imply effective defense.
- e) **Recover:** Plans recovery actions for business continuity.
 - Subnet Service Continuity C_{svc} : Includes RTO and RPO policies, setting business recovery priorities.
 - Isolated Node Count: Indicates the severity of service disruption.
 - Connectivity: Measures the impact of isolation on availability. For a subnet $G'_t = (V', E')$, connectivity is the ratio of actual to maximum theoretical edges:

$$\text{Conn}(G'_t) = \frac{|E'_t|}{|V'| \times (|V'| - 1) / 2}. \quad (9)$$

The above contextual information and key metrics are concatenated via prompts to form the natural language observation information O_t^{LLM} . To ensure the reproducibility of our study, the complete collection of prompt templates utilized in the perception module is disclosed in supplemental files.

3) *Memory:* To cope with the dynamic changes in attack strategies and scales, the LLM agent requires the capability to remember and analyze past attack events, enabling long-term planning and multi-attack-chain analysis across time steps. This memory-driven adaptability is essential for the framework's robustness, as it allows the agent to identify and counter evolving threats in real-time without requiring retraining on new attack patterns. In LLM-based agent frameworks, Memory refers to mechanisms maintaining historical interactions to support current decisions. This paper designs two memory mechanisms: Long-Term Memory and Short-Term Memory, detailed as follows:

- a) **STM:** STM maintains immediate context within the most recent decision cycle, containing the agent's previous action and environmental feedback. Specifically, STM at time t is a triple:

$$STM_t = (a_{t-1}^{LLM}, O_{t-1}^{LLM}, s_t). \quad (10)$$

STM ensures coherent reasoning trajectory generation, allowing the LLM to understand environmental changes caused by its previous actions. This supports continuous

strategy optimization and avoids myopic reasoning or repeated decisions due to information fragmentation.

- b) **LTM**: LTM acts as an external knowledge base for persistently storing and analyzing attack chains. To ensure scalability and prevent context overflow during long-term campaigns, LTM employs a sliding temporal window W to retain only active trajectories. At time t , LTM_t is defined as the set of attack chains occurring within the interval $[t - W, t]$:

$$LTM_t = \{AC_1, AC_2, \dots, AC_k \mid \forall (n, \tau) \in AC_i, \tau \in [t - W, t]\}. \quad (11)$$

Each attack chain AC_i is represented as a sequence of compromised events within the subnet G'_i :

$$AC_i = \left(G'_i, \left[(n_{i,1}^{\text{target}}, t_{i,1}), (n_{i,2}^{\text{target}}, t_{i,2}), \dots \right] \right). \quad (12)$$

Here, each tuple $(n^{\text{target}}, \tau)$ records a target node n^{target} compromised at time τ . By discarding historical data outside the window W , this mechanism ensures that the memory size remains bounded and the inference efficiency is preserved regardless of the attack duration.

Algorithm 2 Reactive Retrieval and Analysis of LTM

- 1: **Input**: Attack chains L_c , Long-term Memory G_p , Critical distance threshold θ , Top-k similarity threshold δ
 - 2: **Output**: Predicted attack chain L_p
 - 3: **if** CriticalDistance $< \theta$ **then**
 - 4: $L_p \leftarrow \emptyset$
 - 5: Return L_p
 - 6: **end if**
 - 7: **for** L_i where $L_i \in G_p$ **do**
 - 8: Match score $M_s \leftarrow \text{similarity}(L_c, L_i)$
 - 9: **end for**
 - 10: Rank L_i by M_s in descending order and select the Top-k chains
 - 11: Calculate $L_p \leftarrow \text{most_likely_attack_chain}(\text{Top-k})$
 - 12: Select L_p
 - 13: Append L_p
 - 14: Integrate L_p into LLM observation O for tactical planning
-

LTM retrieval is reactive. When the Critical Distance of a subnet falls below a threshold θ_{critical} , the mechanism retrieves and analyzes Top-k similar chains from the memory bank, integrating the most probable attack chain into the LLM's observations. The specific retrieval and analysis algorithm is provided in Algorithm 2.

4) *Action and Tools*: The upper-layer LLM agent translates defense decisions into specific defense instructions and actions through the Action and Tools module. This module provides the following two types of tools:

- a) **Action Execution Functions**: These functions enable the LLM agent to directly execute defense actions on network nodes. An action execution function is defined as `ExecuteAction(operation, n)`, where the input consists of the action type and the target node.
- b) **Lower-layer Agent Assignment Tools**: These tools are used to dispatch lower-layer RL agents to specific subnets. A tool can be defined as `AssignAgent( $\phi$ ,  $G'$ )`,

where ϕ represents the type of RL agent and G' denotes the target subnet. Within any single time step t , the usage of this tool by the LLM agent is constrained by the total global agent count.

Through the design of the tools, the proposed framework achieves flexible integration of tactical planning and defense action execution. This design allows the upper-layer LLM to conduct tactical planning via the ReAct paradigm and orchestrate functionally heterogeneous lower-layer RL agents according to the real-time offensive and defensive situation. As a result, the framework combines the LLM's capabilities in planning and generalization with the specialized execution strengths of RL agents, enabling the dynamic generation and implementation of customized defense solutions tailored to specific attack strategies and stages.

IV. EXPERIMENTS

A. Experimental Design

To systematically evaluate the effectiveness of the proposed CyberOps-Bots framework, the following research questions (RQs) are designed:

- **RQ1**: How robust is the CyberOps-Bots framework when facing dynamic changes in cloud network structure (A1)?
- **RQ2**: How robust is the CyberOps-Bots framework when facing dynamic changes in cloud node scale (A2)?
- **RQ3**: Is the CyberOps-Bots framework robust against unseen and diverse cloud attack policies (A3)?
- **RQ4**: Can the CyberOps-Bots framework effectively handle changes in attack scale caused by an increasing number of concurrent attackers (A4)?
- **RQ5**: Can the CyberOps-Bots framework balance defense effectiveness and network availability to enhance cyber resilience?
- **RQ6**: Can the CyberOps-Bots framework improve long-term network resilience through proactive hardening?
- **RQ7**: How is the decision-making efficiency and reliability of the CyberOps-Bots framework?
- **RQ8**: Can the CyberOps-Bots framework effectively support HITL?

This study designs experimental scenarios based on the AWS enterprise cloud dataset [49]. The scenario configurations are dynamically switched to validate A1-A4, as shown in Fig.4. The network scale, structure, exposure surface, asset distribution, vulnerability configuration, and service continuity requirements of the scenarios are designed based on the metadata and attack log data from this dataset. The dataset is also used to summarize the network context C for the LLM, as described in Section III-D2. Specifically:

- **Scale and Structure**: Mirroring the dataset's infrastructure, the experimental network comprises 450 nodes divided into 6 subnets (5 departments and a server room). The node count in each subnet corresponds to the actual device distribution of the respective departments.
- **Network Exposure**: Gateway nodes and inter-subnet topology are configured according to the dataset's connection and isolation policies, aligning east-west traffic with real segmentation requirements. These boundary

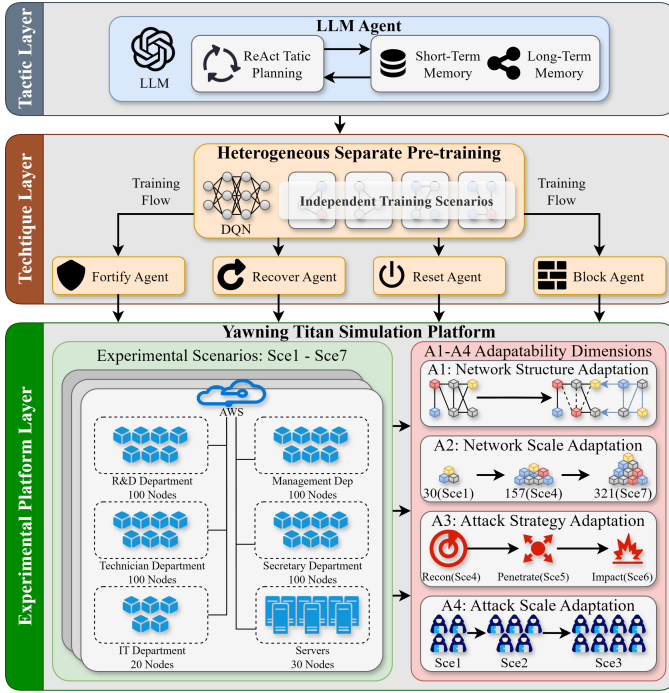


Fig. 4. Experimental setup for evaluating the adaptability of CyberOps-Bots and baseline algorithms.

definitions and connection policies constitute the context C_{exp} .

- **Assets:** HVNs, such as Web and Database servers, are designated based on departmental business functions. These business descriptions and HVN distributions are summarized as the asset context C_{ast} .
- **Vulnerability:** Nodes are assigned vulnerability values based on the dataset's specific attack types and CVE records. This information is summarized as the vulnerability context C_{vul} .
- **Service Continuity:** Operational policies for Recovery Time/Point Objectives (RTO/RPO) are derived from the dataset's business metadata and service continuity agreements, summarized as the service continuity context C_{svc} .

The complete experimental subnet configurations are shown in Table II.

To account for variations in attack strategies and phases [22], this paper designs three types of attack policies based on the scenario envisioned in the introduction. These correspond to different ATT&CK Tactics [18]. The first is **Recon**, aligning with the Reconnaissance and Discovery tactics. Its logic involves attackers randomly probing reachable nodes to assess the attack surface. The second is **Penetrate**, corresponding to the Initial Access and Lateral Movement tactics, where attackers prioritize targeting nodes with higher vulnerability values to achieve infiltration and spread within the network. The third is **Impact**, associated with the Collection and Impact tactics, where attackers plan optimal paths based on network information to directly target high-value nodes, aiming to cause service disruption or data theft.

Different scenarios are switched during the experiments to validate the adaptability of various methods. The LLM used in

this paper is Qwen3-8B [50]. The experimental scenarios are built upon the cyber defense simulation environment Yawning Titan [44], and the baseline algorithms are implemented based on XuanCe [51]. The simulation environment can serve as a benchmark for trajectory comparison. The framework calculates expected environment state based on the RL agents' policies. Comparing this state with the actual state from human intervention quantifies the discrepancy between actual tactical commands and the RL policy. These deviations are recorded in auditable decision logs, providing a data foundation for analyzing human expertise and iteratively refining the collaborative strategy. The configured experimental scenarios are shown in Table III.

Following ISO/IEC 27001:2022 security domains (Protect, Resilience, Defend) [46], this paper develops four heterogeneous lower-layer RL agents. The Fortify Agent hardens networks via Patch actions (Protect) to minimize attack surfaces. The Recover Agent uses Recover actions (Resilience) to restore services post-isolation, ensuring continuity. The Purge Agent integrates Image Reset and Patch actions (Defend) to eliminate threats and rebuild nodes. The Block Agent employs Isolate operations (Defend) to sacrifice connectivity for asset protection and attack containment.

This paper employs the DQN algorithm to train the RL agents. Notably, our hierarchical framework is modular and algorithm-agnostic, allowing for the seamless replacement of DQN with other RL algorithms in future usage. The DQN network used is a two-layer fully connected neural network, with each hidden layer containing 64 nodes. Other experimental settings are detailed in Table IV.

The selected baseline algorithms include IPPO [52], IQL [53], MAPPO [52], QMIX [54], and VDN [55].

B. RQ1: Robustness against Dynamic Network Structure

This experiment validates the adaptability of the proposed framework in scenarios with highly dynamic changes in network structure.

The experiment is conducted based on Sce1. During the experiment, at random intervals, configurations including entry nodes, HVNs, and node vulnerabilities are randomly adjusted, while some nodes are randomly isolated, thereby creating a network scenario characterized by high uncertainty and dynamic changes. Algorithm performance is evaluated using the mean reward, the coefficient of variation of the reward, the mean healthy node ratio, and the mean episode length. Here, the coefficient of variation of the reward is the ratio of the standard deviation to the mean of the average reward within 30 time steps after a dynamic change in the network, reflecting its performance fluctuation when the network structure changes dynamically. The healthy ratio refers to the proportion of nodes that are neither compromised nor isolated. The experimental results are shown in Table V.

Our method significantly outperforms the baseline algorithms in terms of mean reward, mean healthy node ratio, and mean episode length. The significantly higher mean healthy node ratio demonstrates the framework's resilience, as it successfully maintains a larger proportion of available services despite the continuous structural disruptions (A1).

TABLE II
SUBNET CONFIGURATIONS OF THE EXPERIMENTAL CLOUD NETWORK.

Subnet Name	Node Scale	Service	Key Assets	Entry nodes
Dep1	100	Research and Development department. This subnet generates internal R&D traffic and accesses internal servers. Hosts various client workstations used by researchers.	- R&D Workstation - Internal Code Repository Server	5
Dep2	100	Management Department. Handles administrative functions and internal communications. Traffic may include access to financial records and management systems.	- Management Workstation - Internal File Share Server	6
Dep3	100	Technician Department. Responsible for technical support and infrastructure monitoring. May have higher network privileges and generate different traffic patterns.	- Technician Workstation - Network Monitoring Server	4
Dep4	100	Secretary and Operation Department. Handles day-to-day operational tasks, external communications, and customer-facing activities.	- Patch Management Server - Customer Relations Management (CRM) Server - Email Server	5
Dep5	20	IT Department. This subnet manages IT infrastructure and has access to core network services.	- IT Administrator Workstation - DNS Server	3
Servers	30	Central server room hosting critical services for the entire organization. Includes web servers, database servers, and other infrastructure servers.	- Apache Web Server - MySQL Database Server - Windows Server 2012/2016 (For various enterprise services)	2

TABLE III
EXPERIMENTAL SCENARIOS.

Scenario Name	Node Scale	Subnets in Use	Attack Setting		Validation Objective
			Attacker Scale	Attack Policy	
Sce1	30	Servers	6	recon	A1, A2, A4
Sce2			7		A4
Sce3			8		A4
Sce4	150	Servers, Dep1, Dep2	6	recon	A2, A3
Sce5				penetrate	A3
Sce6				impact	A3
Sce7	450	Servers, Dep1, Dep2, Dep3, Dep4, Dep5	6	recon	A2

TABLE IV
CONFIGURATION FOR DQN TRAINING OF LOWER-LAYER RL AGENTS.

Parameter	Value
Learning rate	0.01
Batch size	256
Discount factor γ	0.8
Buffer size	100,000
Optimizer	Adam
Episodes	10,000
Activation function	ReLu
Target update interval	1000
ϵ -greedy exploration factor	$\epsilon_{start} = 0.6, \epsilon_{end} = 0, p = 2000$
CPU	16 $\times$ AMD EPYC 7453
GPU	NVIDIA RTX4090
Hard disk	700GB
Memory	56GB
Operating System	Ubuntu 20.04.1

TABLE V
EXPERIMENTAL RESULTS UNDER SCENARIO 1.

Metrics	Ours	IPPO	IQL	MAPPO	QMIX	VDN
Mean Reward	-0.67	-3.44	-3.64	-3.17	-3.65	-3.68
Reward Coefficient	0.011	0.059	0.037	0.057	0.029	0.028
Variations						
Mean Healthy Ratio	0.91	0.84	0.59	0.72	0.60	0.61
Mean Episode Length	97.3	44.1	23.9	41.2	20.6	19.9

C. RQ2: Robustness against Dynamic Network Scale

To validate the adaptability of the proposed method to changes in cloud network structure and scale, this section configures experimental scenarios that dynamically transition from Sce1 to Sce4 and Sce7, thereby increasing the network node scale from 30 to 450. The adaptability to dynamic changes in network structure and scale is evaluated by comparing the average cumulative reward and jumpstart performance [56], [57] of different algorithms. Here, jumpstart, calculated as defined in [56], refers to the average cumulative reward obtained by an agent during the first 10 episodes after training begins in a new environment. This metric reflects the algorithm's early performance during network dynamic changes, indicating its capability for rapid adaptation to dynamically changing environments.

As shown in Fig.5(a)–(c), during the dynamic transition from Sce1 to Sce4 and then to Sce7, the average cumulative reward of our method is significantly higher than those of baseline algorithms. Furthermore, the reward curves converge quickly after each scenario switch and remain stable after 10,000 steps. Particularly in the Sce7 scenario (Fig.5(c)), where the node scale expands from 30 to 450, although the reward values of IPPO and MAPPO gradually approach our method after 80,000 steps, their convergence is slow.

Fig.5(d) shows the jumpstart performance of each algorithm

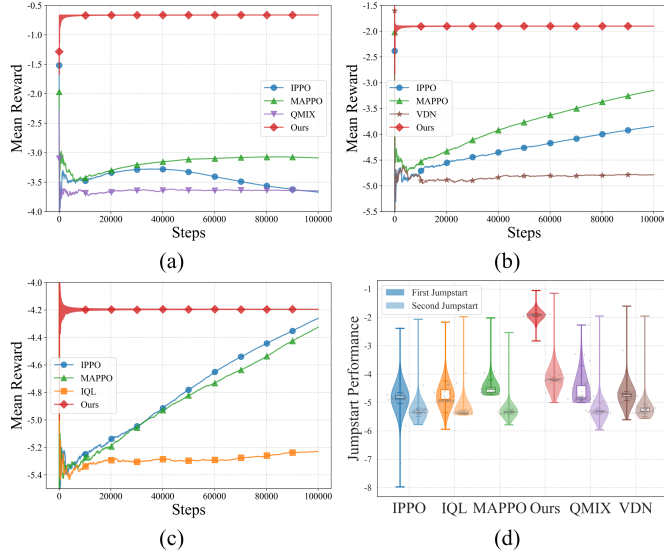


Fig. 5. Figure (a-c) present the experimental results when the test scenario is dynamically switched from Sce1 (a) to Sce4 (b) and Sce7 (c), showing the variation of average cumulative rewards over decision steps. Figure (d) illustrates the jumpstart performance of each algorithm during scenario switching, highlighting their ability to adapt quickly to new environments.

during the two dynamic transitions, reflecting their ability to quickly leverage prior experience to adapt to the cloud network and initialize strategies. Our method achieves the best jumpstart performance, with its values being concentrated and overall higher than those of the baseline algorithms. This indicates that the method can rapidly respond to changes in network structure and scale, maintaining high defense effectiveness without retraining. This is because the upper-layer LLM agent perceives the network situation via natural language descriptions (Section III-D2), mapping diverse network states into a unified IPDRR-based semantic space, while the lower-layer RL agents only need to focus on local network states. Consequently, when the network scale expands, the CyberOps-Bots framework effectively avoids the problem of state space explosion, achieving seamless adaptation to the scaling of the cloud network (A2).

D. RQ3: Robustness against Evolving Attack Policy

To validate the adaptability of the proposed method to different attack policies/phases, this section configures the experimental scenarios to dynamically switch from Sce4 to Sce5 and Sce6, thereby changing the attack policy from Recon to Penetrate and Impact.

As shown in Fig.6(a)-(c), during the dynamic transition of the attack strategy, the average cumulative reward of our method consistently and significantly outperforms the baseline algorithms. The jumpstart results in Fig.6(d) shows that our method achieves the highest average jumpstart value during both dynamic transitions. This validates that the framework, through its heterogeneously separated pre-trained lower-layer RL agents (see Section III-C2), can dynamically combine diverse defense strategies via the LLM's tactical planning, thereby handling unknown and evolving attack policies (A3).

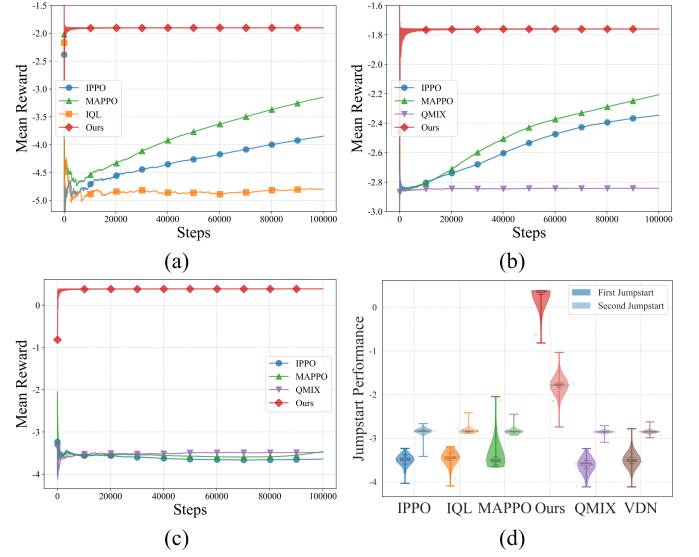


Fig. 6. Figure (a-c) present the experimental results when the test scenario is dynamically switched from Sce4 (a) to Sce5 (b) and Sce6 (c), showing the variation of average cumulative rewards over decision steps. Figure (d) illustrates the jumpstart performance of each algorithm during scenario switching, highlighting their ability to adapt quickly to new environments.

E. RQ4: Robustness against Evolving Attack Scale

To validate the proposed method's adaptability to dynamic attack scale, this section configures experimental scenarios that transition from Sce1 to Sce2 and Sce3. This setup increases the number of attackers from 6 to 8, thereby characterizing the growth in attack scale by escalating concurrent threats.

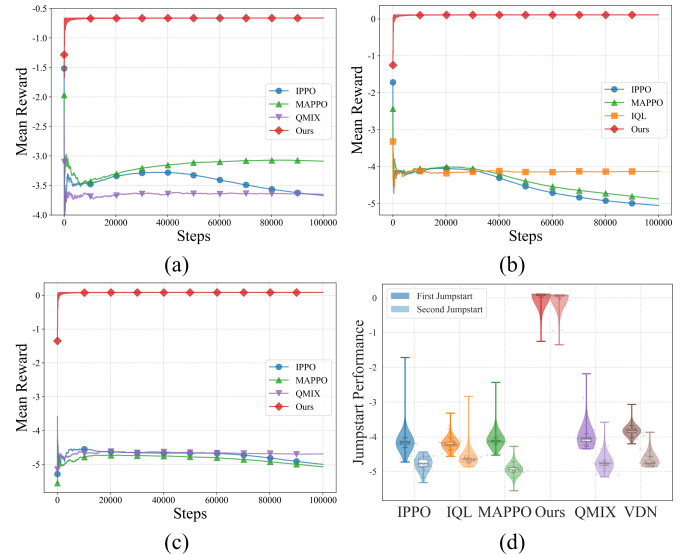


Fig. 7. Figure (a-c) present the experimental results when the test scenario is dynamically switched from Sce1 (a) to Sce2 (b) and Sce3 (c), showing the variation of average cumulative rewards over decision steps. Figure (d) illustrates the jumpstart performance of each algorithm during scenario switching, highlighting their ability to adapt quickly to new environments.

As can be seen from Fig.7(a)-(c), the proposed method consistently maintains a leading position in terms of average cumulative reward. Particularly in Sce3, where the number of

attackers doubles that of the defenders, baseline algorithms such as IPPO and IQL exhibit a notable decline in cumulative reward, while the proposed method still sustains a relatively high reward level. The jumpstart results in Fig.7(d) indicate that the proposed method achieves the highest jumpstart values during both transitions in attack scale, demonstrating its capability to quickly adapt to dynamic attack scale (A4).

To investigate whether the variations in attack policies affect the robustness of defense frameworks under dynamically changing attack scales, we extend the aforementioned experiments that dynamically shifted attack scales across scenarios Sce1, Sce2, and Sce3. Specifically, we further replace the base attack policy (Recon) with the more threatening Penetrate and Impact policy, respectively, to examine the trends in jumpstart performance for different algorithms as the attack scale increases under these advanced adversarial behaviors. Experimental results are presented in Table VI.

TABLE VI
JUMPSTART UNDER VARIOUS ATTACK POLICIES AND SCALES.

Algorithm	Attack Strategy			
	<i>Penetrate</i>		<i>Impact</i>	
	Attack Scale			
	$6 \rightarrow 7$	$7 \rightarrow 8$	$6 \rightarrow 7$	$7 \rightarrow 8$
IPPO	-4.42	-5.56	-5.95	-6.51
IQL	-4.38	-4.91	-5.48	-5.87
MAPPO	-4.24	-4.95	-5.12	-5.89
QMIX	-4.55	-5.62	-6.13	-6.73
VDN	-4.48	-5.47	-5.85	-6.92
Ours	0.12	-0.24	-2.05	-2.22

The data analysis reveals that regardless of how the attack policy evolves, a dynamic increase in attack scale leads to a decline in the jumpstart performance of all baseline algorithms. In contrast, the proposed CyberOps-Bots framework consistently maintains the best jumpstart performance under all tested conditions. This finding indicates that the evolution of attack policy further impairs the ability of baseline algorithms to adapt to dynamic attack scales. The proposed framework, however, suffers relatively less from this effect due to its long-term memory mechanism, which enables continuous tracking and analysis of multiple attack chains. Consequently, even when confronted with complex threats characterized by the dual dynamics of evolving strategies and scales, the framework retains high robustness and rapid adaptation capability.

F. RQ5: Cyber Resilience through Balancing Defense Effectiveness and Availability

This set of experiments validates the proposed framework's capability to balance long-term defense effectiveness with network availability protection during dynamic network changes. We analyze the relationship between the Maximum Episode Length and the Mean Healthy Ratio for all algorithms across the dynamic scenarios described in the preceding RQs.

Specifically, Fig.8(a-c) presents the results when the network scale dynamically expands from Sce1 (30 nodes) to Sce4 (150 nodes) and Sce7 (450 nodes). Fig.8(d-f) corresponds to

scenarios where the attack strategy switches from Sce4 (recon) to Sce5 (penetrate) and Sce6 (impact). Fig.8(g-i) reflects the algorithmic performance as the attack scale gradually increases from Sce1 (6 attackers) to Sce2 (7 attackers) and Sce3 (8 attackers).

As shown in Fig.8(a-c), with dynamic changes in network scale, the maximum episode length of baseline algorithms decreases significantly. Furthermore, maintaining a longer episode often requires sacrificing network availability, as seen in Fig.8(c). In contrast, our method maintains the highest episode length (100) and health ratio (average 0.87) throughout the dynamic transition from Sce1 to Sce7.

As shown in Fig.8(d-f), our method consistently achieves the highest episode length and a high network healthy ratio. In comparison, baseline algorithms tend to over-isolate nodes when countering the penetrate strategy, leading to a substantial drop in healthy ratio (Fig.8(e)). Under the impact strategy (Fig.8(f)), the episode length shortens significantly due to the inability to effectively block concentrated attacks on high-value nodes.

As depicted in Fig.8(g-i), when the attack scale grows dynamically, our method maintains the highest episode length (100) and a relatively high average health ratio (average 0.84 in Sce3). Conversely, the performance of baseline algorithms declines markedly after the attack scale increases, with both their maximum episode length and health ratio severely degraded.

G. RQ6: Resilience Enhancement through Proactive Network Hardening

Proactive hardening refers to the capability of defenders to not only execute effective defense strategies upon the occurrence of a network attack but also proactively patch the network's vulnerabilities based on the incident, thereby preventing future malicious activities. To verify whether each algorithm can effectively perform hardening measures while defending, we statistically analyzes the average network vulnerability (see Definition 3) at the end of each episode across all the aforementioned scenarios.

Fig.9 illustrates the distribution of the average network vulnerability at the end of episodes for algorithms across all the experimental scenarios. The box corresponding to our method is positioned significantly lower than those of the baseline algorithms, and its distribution range is more concentrated. This indicates that our approach can consistently and stably reduce network vulnerability under dynamic conditions. In contrast, the vulnerability distributions of the baseline algorithms are more dispersed and contain more outliers, reflecting the instability of their strategies in achieving network hardening effects within dynamic environments. This result further validates that the CyberOps-Bots framework not only maintains immediate defense effectiveness against dynamic attack threats but also achieves long-term resilience reinforcement.

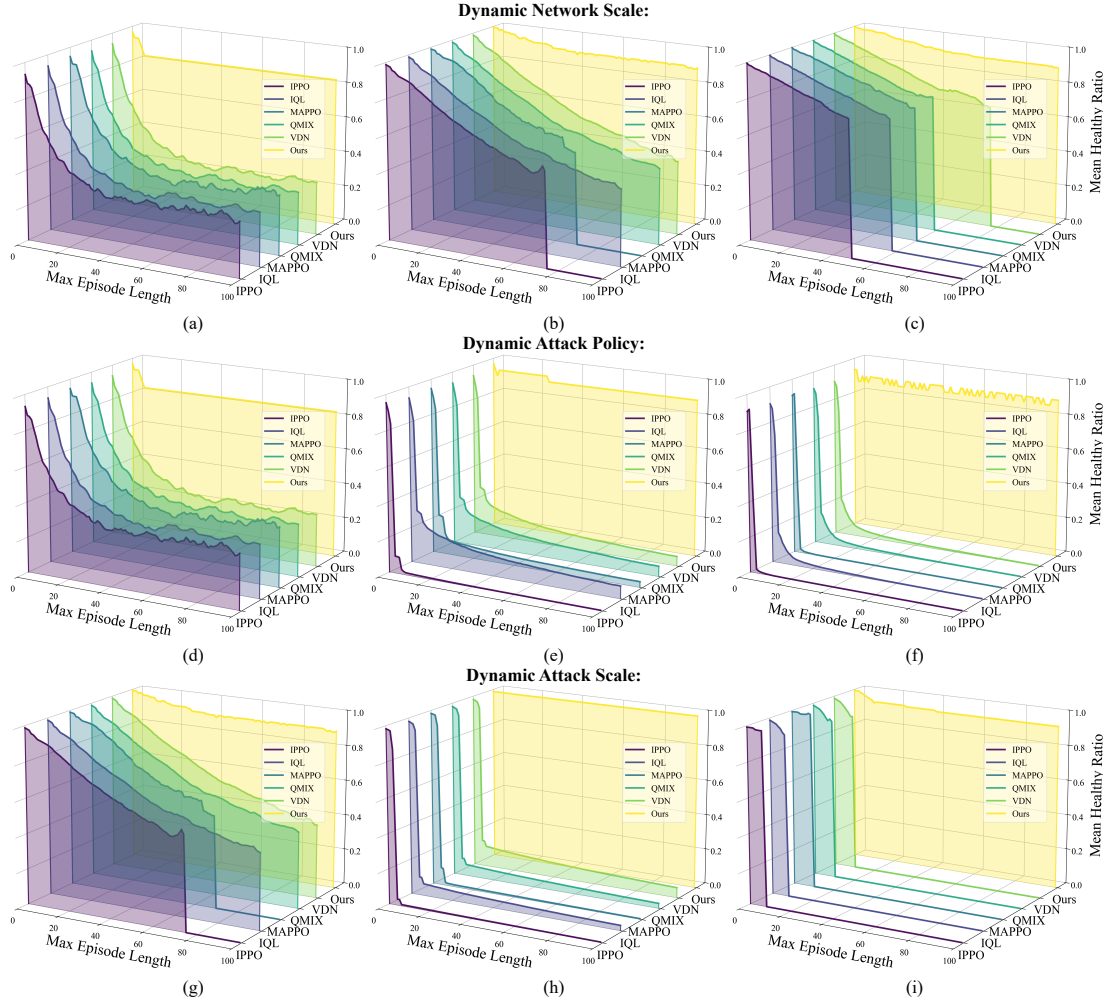


Fig. 8. Comparison of the trade-off between defense persistence (Maximum Episode Length) and resilience (Mean Healthy Ratio) across different algorithms under three aspects: (a)-(c) dynamic network scale changes; (d)-(f) varying attack policies; and (g)-(i) increasing attack scales.

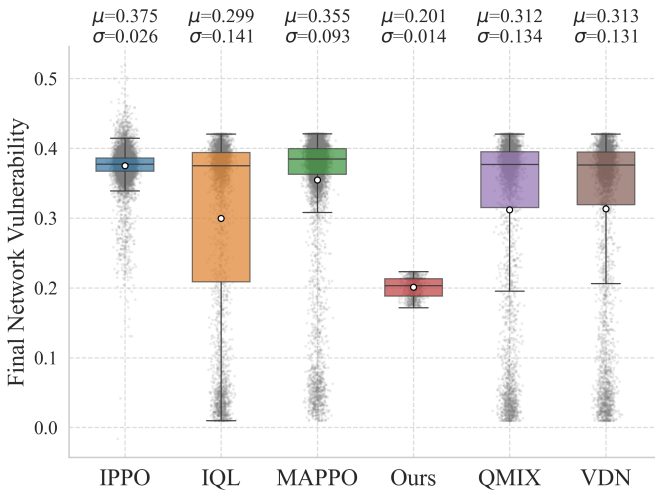


Fig. 9. This figure shows the average network vulnerability value at the end of each episode across all scenarios, reflecting the algorithms' ability to defend against attacks while strengthening the network.

TABLE VII
COMPUTATIONAL EFFICIENCY OF THE LLM AGENT.

Scenario	Node Scale	Token(K)/Step	Time(ms)/Step	
			Ours	Baselines (AVG)
Sce1	30	1.64	335.12	72.09
Sce4	150	2.51	347.85	97.91
Sce7	450	3.62	366.43	141.54

H. RQ7: Operational Robustness Analysis regarding Efficiency and Reliability

This section evaluates the computational overhead and decision reliability of the proposed framework. Table VII details the computational efficiency metrics across different network scales, including average token consumption and a contrast of our framework's decision time against the average time of all baseline algorithms. Table VIII compares the hallucination rates of the complete framework against ablation configurations.

In this experiment, the LLM's "hallucination" is defined based on the potential risks it introduces during the decision-

TABLE VIII
RELIABILITY ANALYSIS OF THE FRAMEWORK VS. ABLATED VARIANTS.

Framework Name	Hallucination Rate		Defense Effectiveness	
	Reasoning	Final Answer	Mean Episode Length	Mean Healthy Ratio
Ours	8.90%	0.71%	98.91	89.72%
Ours w/o ReAct	8.52%	8.52%	87.13	83.95%
Ours w/o STM	10.03%	1.64%	84.22	81.04%
Ours w/o LTM	8.91%	0.71%	66.87	69.08%

making process, including two categories: The first category is factual hallucination, where the content generated by the LLM contradicts the specific cloud network context provided by the perception module. The second category is formatting hallucination, where the LLM’s output fails to strictly adhere to formatting requirements or tool-calling interface specifications. This includes errors in parameter types, out-of-range values, or structures incompatible with executable command requirements. Our formatting requirements for output align with standards of the OpenAI API formatting guidelines [58]. The hallucination rate, which quantifies decision-making reliability, is calculated by counting the occurrences of the aforementioned issues over a defined number of decision steps.

All data are averaged over 1000 decision steps across all scenarios in Table III.

Experimental results indicate that token consumption and decision time are primarily correlated with network scale. As shown in Table VII, token usage increases linearly as the network scale doubles. Given the context window capabilities of current mainstream LLMs, the framework demonstrates robust token efficiency and scalability. Regarding decision latency, by leveraging EAGLE-3 [59] for inference acceleration and imposing strict constraints on LLM reasoning and output formats, our framework achieves millisecond-level decision times per step. Although this inference speed is naturally slower than lightweight RL baseline algorithms, the latency remains within an acceptable range considering the framework’s superior performance. Notably, as the network scale expands, the time consumption of baseline algorithms exhibits a sharper increase due to the escalating computational complexity caused by the multiplication of input states. In contrast, our framework’s decision time increases only marginally. This is because the increased scale primarily translates to more input tokens—which remain far below the maximum context limit—while the strict prompt constraints ensure that the time required for generation remains largely independent of network scale.

As presented in Table VIII, while the framework exhibits an 8.90% hallucination rate during the reasoning process, the multi-step verification mechanism of the ReAct paradigm significantly reduces the final decision hallucination rate to 0.71%. When the ReAct module is removed, although the single-step reasoning hallucination drops slightly (8.52%), the lack of iterative correction causes the final answer hallucination rate to spike, exceeding that of the complete framework. Furthermore, removing the STM module leads to a loss of decision context, increasing the reasoning hallucination rate to 10.03%. While removing the LTM module has a negligible

impact on hallucination rates, it substantially undermines defense effectiveness, as evidenced by the significant drops in both episode length and healthy ratio. In conclusion, the ReAct planning and memory mechanisms significantly improve the decision reliability of the framework.

I. RQ8: Support Capability for Human-in-the-Loop

To evaluate the functional efficacy of the HITL mechanism, this section injects security expert instructions to test the framework’s adaptability to new tactical objectives, analyzing how human intervention shifts the agent’s decision-making logic. We also analyze the interpretability and auditability of the LLM agent’s reasoning chain to evaluate the transparency of human-machine collaborative decision-making.

By default, the defense strategy prioritizes high-value asset protection, treating the service continuity of all subnets with equal importance. In this experiment, the expert instruction in Fig.10 was injected into the LLM agent via the perception module to override the default priorities.

This instruction imposes a higher-level objective over the predefined reward function, requiring the framework to prioritize the availability of the server subnet. Upon receiving this instruction, the LLM generated reasoning chain shown in Fig.10 over three time steps.

The sequential reasoning chain across these three time steps demonstrates the tactical evolution of the LLM agent under human intervention:

- **T0 (Preventive Defense):** Prioritizes hardening the core server subnet in the absence of immediate threats.
- **T1 (Threat Response):** Balances multiple threats while maintaining core service continuity as the primary objective.
- **T2 (Precise Isolation):** Executes targeted isolation based on predicted attack paths, minimizing service interruption costs while conducting image reset and recovery.

Experimental results indicate that the LLM agent effectively comprehends and executes high-level objectives prioritizing core server continuity.

Following the human instruction, the reasoning chain at T1 and T2 shows a deliberate trade-off: the agent accepted a higher risk in other subnets to ensure absolute availability of the core server subnet. This alignment between human intent and the agent’s subsequent reasoning logic provides empirical evidence of the HITL mechanism’s functional effectiveness.

To evaluate the practical efficacy of the HITL mechanism in guiding defense decisions, we report average results from 1,000 decision steps across all scenarios listed in Table III.

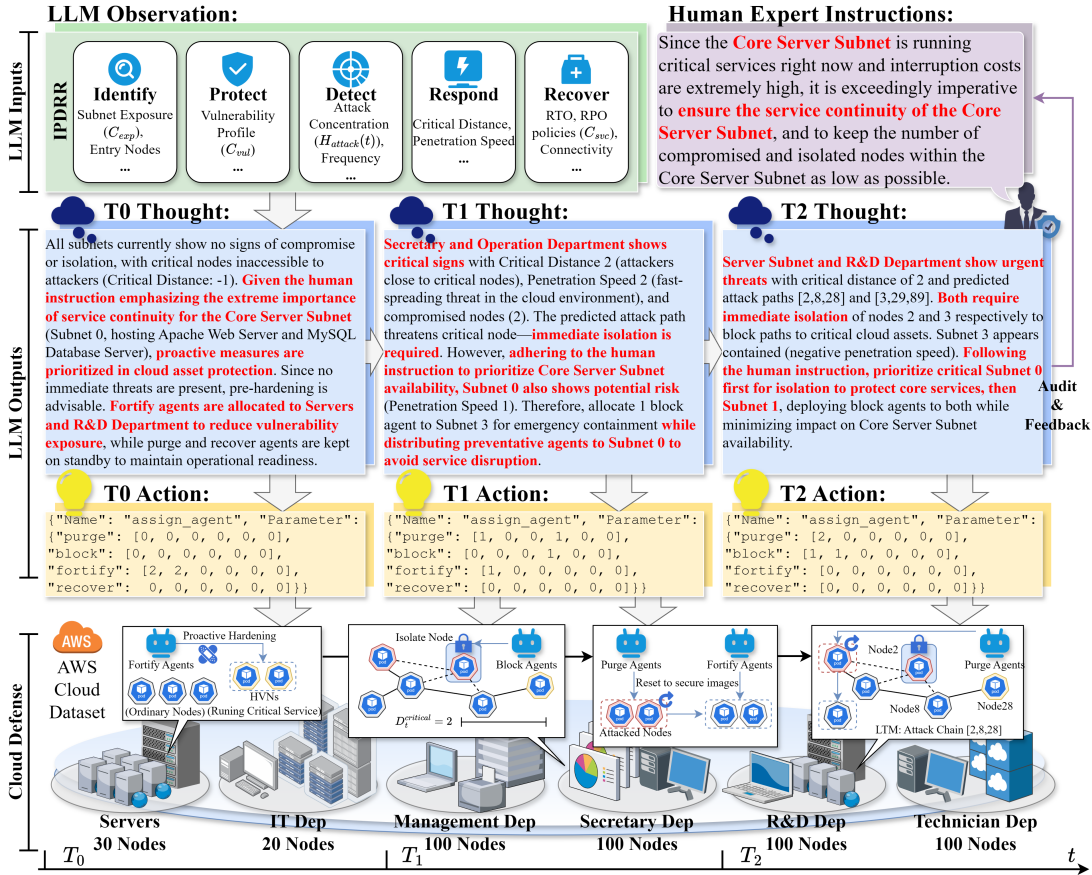


Fig. 10. Illustration of the LLM agent’s tactical evolution under human intervention. The diagram displays the reasoning chain and defense actions across three time steps (T0, T1, and T2) in response to expert instructions.

TABLE IX
QUANTITATIVE EVALUATION OF HITL EFFECTIVENESS ON DEFENSE DECISION-MAKING.

Configuration	Core Server Subnet	Overall Metrics	
	Subnet Mean Healthy Ratio	Mean Healthy Ratio	Mean Episode Length
With Human Instructions	91.07%	90.36%	95.88
Without Human Instructions	84.83%	89.72%	98.91

The evaluation metrics encompass the healthy node ratio of the core server subnet specified by the human instruction, as well as the overall network-wide healthy node ratio and average episode length. The quantitative results are summarized in Table IX. The analysis reveals that after injecting the human instruction to “prioritize the availability of the server subnet”, the framework achieves a significantly higher healthy node ratio for the targeted subnet, demonstrating a clear improvement over the baseline without instruction. This indicates that the upper-layer LLM agent can accurately comprehend the high-level tactical intent conveyed via natural language and dynamically adjust its defensive focus accordingly. It is noteworthy that while executing this focused adjustment, the framework’s overall defensive efficacy remains intact. The results confirm that the designed HITL mechanism can effectively and flexibly integrate human expert knowledge and instructions into the automated decision cycle.

Furthermore, the ReAct-based reasoning log explicitly records the rationale behind tactical decisions and the integration of human instructions. This transparency provides security experts with an audit trail for real-time supervision and post-incident review, thereby enhancing the reliability and trustworthiness of human-machine collaboration in complex cloud defense scenarios. Crucially, these HITL auditing and inspection procedures provide essential oversight, effectively mitigating potential risks caused by LLM hallucinations.

V. CONCLUSION

This paper proposes CyberOps-Bots, a hierarchical multi-agent RL framework for cloud network resilience. By synergizing LLM-based high-level planning with atomic RL execution, the framework effectively addresses critical adaptability challenges in dynamic network environments. Experimental results demonstrate a 68.5% improvement in network availability and a 34.7% jumpstart gain without retraining. Notably,

this is the first framework to offer inherent Human-in-the-Loop support and superior interpretability. While the centralized LLM architecture poses potential scalability and security constraints, future work will focus on decentralized multi-LLM coordination, integrating model security safeguards, and designing cross-layer decision verification mechanisms to enhance the security of the autonomous defense ecosystem. Furthermore, investigating automated prompt optimization and sensitivity analysis will be prioritized to further bolster the framework's robustness against semantic variations. Overall, CyberOps-Bots represents a significant advancement toward autonomous, adaptive, and sustainable cloud defense.

ACKNOWLEDGMENTS

This work was supported in part by the National Natural Science Foundation of China under Grants 61902427 and 62471064, and by the National Key Research and Development Program of China under Grants 2023YFC3306305 and 2022YFB2902200.

REFERENCES

- [1] R. Dilworth, "Advancements and challenges in cloud computing: Multi-cloud management, security, and ai-driven threat mitigation," in *Proceedings of the 2024 7th Artificial Intelligence and Cloud Computing Conference*, 2024, pp. 639–645.
- [2] D. Soldani, P. Nahi, H. Bour, S. Jafarizadeh, M. F. Soliman, L. Di Giovanna, F. Monaco, G. Ognibene, and F. Risso, "ebpf: A new approach to cloud-native observability, networking and security for current (5g) and future mobile networks (6g and beyond)," *IEEE Access*, vol. 11, pp. 57 174–57 202, 2023.
- [3] L. Rosa, L. Foschini, and A. Corradi, "Empowering cloud computing with network acceleration: A survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 4, pp. 2729–2768, 2024.
- [4] K. Venkata, "Autonomous cloud networking in 2024: Leveraging ai and intent-based architectures for self-healing and optimization," 2025.
- [5] A. M. Abdallah, A. Saif Rashed Obaid Alkaabi, G. Bark Nasser Douman Alameri, S. H. Rafique, N. S. Musa, and T. Murugan, "Cloud network anomaly detection using machine and deep learning techniques— recent research advancements," *IEEE Access*, vol. 12, pp. 56 749–56 773, 2024.
- [6] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2023.
- [7] M. Foley, C. Hicks, K. Highnam, and V. Mavroudis, "Autonomous network defence using reinforcement learning," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ser. ASIA CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1252–1254. [Online]. Available: <https://doi.org/10.1145/3488932.3527286>
- [8] H. Hu, Y. Liu, C. Chen, H. Zhang, and Y. Liu, "Optimal decision making approach for cyber security defense using evolutionary game," *IEEE Transactions on Network and Service Management*, vol. 17, no. 3, pp. 1683–1700, 2020.
- [9] S. Vyas, J. Hannay, A. Bolton, and P. P. Burnap, "Automated cyber defence: A review," *arXiv preprint arXiv:2303.04926*, 2023.
- [10] Q. Li, R. Wang, D. Li, F. Shi, M. Zhang, A. Chattopadhyay, Y. Shen, and Y. Li, "Dyngpen: Automated penetration testing in dynamic network scenarios using deep reinforcement learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 8966–8981, 2024.
- [11] C. Lu, K. Ye, G. Xu, C.-Z. Xu, and T. Bai, "Imbalance in the cloud: An analysis on alibaba cluster trace," in *2017 IEEE International Conference on Big Data (Big Data)*, 2017, pp. 2884–2892.
- [12] C. Reiss, A. Tumanov, G. R. Gangar, R. H. Katz, and M. A. Kozuch, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in *Proceedings of the third ACM symposium on cloud computing*, 2012, pp. 1–13.
- [13] N. Novaes Neto, S. Madnick, A. Moraes G de Paula, and N. Malara Borges, "A case study of the capital one data breach," *Stuart E. and Moraes G. de Paula, Anchises and Malara Borges, Natasha, A Case Study of the Capital One Data Breach (January 1, 2020)*, 2020.
- [14] Google Cloud Armor, "How Google Cloud Blocked Largest Layer 7 DDoS Attack at 46 Million RPS," 2022. [Online]. Available: <https://cloud.google.com/blog/products/identity-security/how-google-cloud-blocked-largest-layer-7-ddos-attack-at-46-million-rps>
- [15] T. Purves, K. G. Kyriakopoulos, S. Jenkins, I. Phillips, and T. Dudman, "Causally aware reinforcement learning agents for autonomous cyber defence," *Knowledge-Based Systems*, vol. 304, p. 112521, 2024.
- [16] T. Zhu, D. Ye, Z. Cheng, W. Zhou, and P. S. Yu, "Learning games for defending advanced persistent threats in cyber systems," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 4, pp. 2410–2422, 2023.
- [17] L. Xiao, H. Liu, Z. Lv, Y. Chen, Z. Lin, and Y. Du, "Reinforcement-learning-based apt defense for large-scale smart grids," *IEEE Internet of Things Journal*, vol. 12, no. 9, pp. 11 917–11 925, 2025.
- [18] B. Al-Sada, A. Sadighian, and G. Oliveri, "Mitre attack: State of the art and way forward," *ACM Comput. Surv.*, vol. 57, no. 1, Oct. 2024. [Online]. Available: <https://doi.org/10.1145/3687300>
- [19] Y. Peng, H. Hu, F. Li, Y. Jiang, J. Tang, and Y. Liu, "Llm4game: Multi-agent reinforcement learning with knowledge injection for dynamic defense resource allocation in cloud storage," *Computer Networks*, p. 111748, 2025.
- [20] Y. Li, Z. Xiang, N. D. Bastian, D. Song, and B. Li, "IDS-agent: An LLM agent for explainable intrusion detection in iot networks," 2025. [Online]. Available: <https://openreview.net/forum?id=uuCcK4cmIH>
- [21] X. Li, X. Hu, and T. Jiang, "Dual-reinforcement-learning-based attack path prediction for 5g industrial cyber-physical systems," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 50–58, 2024.
- [22] A. V. Singh, E. Rathbun, E. Graham, L. Oakley, S. Boboila, A. Oprea, and P. Chin, "Hierarchical multi-agent reinforcement learning for cyber network defense," *arXiv preprint arXiv:2410.17351*, 2024.
- [23] J. Chen, X. Lan, Q. Zhang, W. Ma, W. Fang, and J. He, "Defending against apt attacks in cloud computing environments using grouped multiagent deep reinforcement learning," *IEEE Internet of Things Journal*, vol. 12, no. 12, pp. 19 459–19 470, 2025.
- [24] Y. Cao, K. Liu, Y. Lin, L. Wang, and Y. Xia, "Deep-reinforcement-learning-based self-evolving moving target defense approach against unknown attacks," *IEEE Internet of Things Journal*, vol. 11, no. 20, pp. 33 027–33 039, 2024.
- [25] B. Ren, Y. Tang, H. Wang, Y. Wang, J. Liu, G. Gao, and W. Wei, "A multiagent deep reinforcement learning autonomous security management approach for internet of things," *IEEE Internet of Things Journal*, vol. 11, no. 15, pp. 25 600–25 612, 2024.
- [26] G. Jain, A. Kumar, and S. A. Bhat, "Recent developments of game theory and reinforcement learning approaches: A systematic review," *IEEE Access*, vol. 12, pp. 9999–10 011, 2024.
- [27] S. Rass, S. König, J. Wachter, V. Mayoral-Vilches, and E. Panaousis, "Game-theoretic apt defense: An experimental study on robotics," *Computers & Security*, vol. 132, p. 103328, 2023.
- [28] G. Kong, F. Chen, X. Yang, G. Cheng, S. Zhang, and W. He, "Optimal deception asset deployment in cybersecurity: A nash q-learning approach in multi-agent stochastic games," *Applied Sciences*, vol. 14, no. 1, 2024. [Online]. Available: <https://www.mdpi.com/2076-3417/14/1/357>
- [29] A. S. Mohamed and D. Kundur, "Resilient cyber-physical system honeypots for cyberattacker engagement," *IEEE Transactions on Industrial Informatics*, vol. 21, no. 11, pp. 8585–8595, 2025.
- [30] K. Bitirgen and Ü. B. Filik, "Markov game based on reinforcement learning solution against cyber-physical attacks in smart grid," *Expert Systems with Applications*, vol. 255, p. 124607, 2024.
- [31] T. Ramana, M. Thirunavukkarasan, A. S. Mohammed, G. G. Devarajan, and S. M. Nagarajan, "Ambient intelligence approach: Internet of things based decision performance analysis for intrusion detection," *Computer Communications*, vol. 195, pp. 315–322, 2022.
- [32] W. Soussi, G. Gür, and B. Stiller, "Moving target defense (mtd) for 6g edge-to-cloud continuum: A cognitive perspective," *IEEE Network*, vol. 39, no. 1, pp. 149–156, 2025.
- [33] Y. Hou, G. Han, F. Zhang, and C. Lin, "Enhancing underwater iot security: A collaborative pursuit strategy using multi-agent reinforcement learning," *IEEE Internet of Things Magazine*, vol. 7, no. 5, pp. 112–118, 2024.
- [34] Y. Tang, J. Sun, H. Wang, J. Deng, L. Tong, and W. Xu, "A method of network attack-defense game and collaborative defense decision-making based on hierarchical multi-agent reinforcement learning," *Computers & Security*, vol. 142, p. 103871, 2024.
- [35] G. McDonald, L. Li, and R. A. Mallah, "Finding the optimal security policies for autonomous cyber operations with competitive reinforcement learning," *IEEE Access*, vol. 12, pp. 120 292–120 305, 2024.

- [36] L. Zhang, T. Zhu, F. K. Hussain, D. Ye, and W. Zhou, "A game-theoretic method for defending against advanced persistent threats in cyber systems," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 1349–1364, 2023.
- [37] S. Pateria, B. Subagdja, A.-h. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Comput. Surv.*, vol. 54, no. 5, Jun. 2021. [Online]. Available: <https://doi.org/10.1145/3453160>
- [38] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.
- [39] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [40] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.
- [41] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models," *arXiv preprint arXiv:2410.05229*, 2024.
- [42] J. Sun, Y. Tian, W. Zhou, N. Xu, Q. Hu, R. Gupta, J. Wieting, N. Peng, and X. Ma, "Evaluating large language models on controlled generation tasks," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 3155–3168. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.190/>
- [43] N. I. of Standards and Technology, "NIST Cybersecurity Framework," Nov. 2014. [Online]. Available: <https://www.nist.gov/cyberframework>
- [44] A. Andrew, S. Spillard, J. Collyer, and N. Dhir, "Developing optimal causal cyber-defence agents via cyber security simulation," 2022. [Online]. Available: <https://arxiv.org/abs/2207.12355>
- [45] I. Symes Thompson, A. Caron, C. Hicks, and V. Mavroudis, "Entity-based reinforcement learning for autonomous cyber defence," in *Proceedings of the Workshop on Autonomous Cybersecurity*, ser. AutonomousCyber '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 56–67. [Online]. Available: <https://doi.org/10.1145/3689933.3690835>
- [46] c. ISO/IEC Joint Technical Committee 1, Subcommittee 27 – Information security and privacy protection, "ISO/IEC 27001:2022," Geneva, Switzerland, 2022. [Online]. Available: <https://www.iso.org/standard/27001>
- [47] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *The eleventh international conference on learning representations*, 2022.
- [48] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 8634–8652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [49] Communications Security Establishment (CSE) and The Canadian Institute for Cybersecurity (CIC), "A Realistic Cyber Defense Dataset," 2018. [Online]. Available: <https://registry.opendata.aws/cse-cic-ids2018>
- [50] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [51] W. Liu, W. Cai, K. Jiang, G. Cheng, Y. Wang, J. Wang, J. Cao, L. Xu, C. Mu, and C. Sun, "Xuance: A comprehensive and unified deep reinforcement learning library," 2023. [Online]. Available: <https://arxiv.org/abs/2312.16248>
- [52] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. WU, "The surprising effectiveness of ppo in cooperative multi-agent games," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 611–24 624. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9c1535a02f0ce079433344e14d910597-Paper-Datasets_and_Benchmarks.pdf
- [53] L. Matignon, G. J. Laurent, and N. Le Fort-Piat, "Independent reinforcement learners in cooperative markov games: a survey regarding coordination problems," *The Knowledge Engineering Review*, vol. 27, no. 1, p. 1–31, 2012.
- [54] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020. [Online]. Available: <http://jmlr.org/papers/v21/20-081.html>
- [55] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls *et al.*, "Value-decomposition networks for cooperative multi-agent learning," *arXiv preprint arXiv:1706.05296*, 2017.
- [56] S. Zhou, J. Liu, Y. Lu, J. Yang, Y. Zhang, and J. Chen, "Mind the gap: towards generalizable autonomous penetration testing via domain randomization and meta-reinforcement learning," *Frontiers of Information Technology & Electronic Engineering*, vol. 26, no. 12, pp. 2511–2528, 2025. [Online]. Available: <https://doi.org/10.1631/FITEE.2500100>
- [57] M. E. Taylor and P. Stone, "Transfer learning for reinforcement learning domains: A survey," *Journal of Machine Learning Research*, vol. 10, no. 7, 2009.
- [58] Michelle Pokrass, "Introducing Structured Outputs in the API," Aug. 2024. [Online]. Available: <https://openai.com/index/introducing-structured-outputs-in-the-api/>
- [59] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle-3: Scaling up inference acceleration of large language models via training-time test," 2025. [Online]. Available: <https://arxiv.org/abs/2503.01840>

Yixiao Peng received the B.E. degree in information countermeasure technology from Information Engineering University, Zhengzhou, China, in 2024, where he is currently pursuing the M.D. degree in cyberspace security. His research interests include network security and large language models.

Hao Hu received his Ph.D. degree in the Zhengzhou Information Science and Technology Institute in 2018. He has been an associate professor with the National Digital Switching System Engineering and Technological R&D Center since 2018. His research interests include attack-defense modeling and proactive defense.

Feiyang Li received the B.E. degree in information countermeasure technology from Information Engineering University, Zhengzhou, China, in 2023. He is currently pursuing the Ph.D. degree in cyberspace security. His research interests include reinforcement learning and automated penetration testing.

Xinye Cao received the B.E. degree in electronic and information engineering from Central China Normal University (CCNU), Wuhan, China, in 2020. She is currently pursuing the Ph.D. degree in cyberspace security with the National Engineering Research Center for Mobile Network Technologies, Beijing University of Posts and Telecommunications (BUPT). Her research interests include large AI models for future wireless communication systems and wireless communication security.

Yingchang Jiang is a Ph.D. candidate at the Information Engineering University, Zhengzhou, China. His main research interest is to analyze cyber threat intelligence using LLMs and deep learning techniques.

Jipeng Tang received the B.M. degree in Management Science and Engineering from Information Engineering University, Zhengzhou, China, in 2024, where he is currently pursuing the M.D. degree in cyberspace security. His research interests include network security and knowledge graphs.

Guoshun Nan is currently a Full Professor at the National Engineering Research Center for Mobile Network Technologies, Beijing University of Posts and Telecommunications, Beijing, China. He has a broad interest in multimodal learning, large language models, and 6G network security.

Yuling Liu received his Ph.D. degree from the Institute of Software, Chinese Academy of Sciences in China. He is currently a senior engineer at the Institute of Information Engineering, Chinese Academy of Sciences. His research areas are network security situational awareness, network security, big data analysis, and security measurement and certification.